

# **REGULAR EXPRESSION AND TEXT PROCESSING**

**MOD 02 IAN**

# Regular Expression (Regex)

- Python Regex, or regular expressions, is a sequence of characters used to define a search pattern. It helps us match, search, split, or validate strings based on specific patterns.
- The re module in Python supports regular expressions. It allows us to search for strings using patterns and returns the first match or None if no match is found.

# Regular expressions are useful for:

- Identify patterns in a string/ file.
- Search for a substring in a string.
- Replace a section of a string with another string.
- Validate email format.
- Split a string into substrings.

## Example

```
import re

text = "The rain in Spain"
pattern = "rain"

# Search for the word "rain" in the text
match = re.search(pattern, text)

if match:
    print("Match found!")
else:
    print("No match found.")
```

## Output

```
Match found!
```

# Regex Module in Python

- [Python](#) provides a built-in module called re to work with regular expressions. It helps us search, match, and manipulate text using patterns.
- The Python regular expression module is a powerful tool for handling text. Once we understand its functions and syntax, we can efficiently perform pattern matching and text processing tasks.
- Import the Module
- Start by importing the 're' module to use the Python RegEx function.

code - **import re**

# Basic Functions

- **search()**: It is used to search a string for a match. Once found, it returns the matched object.
- **match()**: It checks if the start of a string matches the pattern.
- **findall()**: It finds all matches in a string and returns a list of them.
- **sub()**: It replaces the matches of the pattern with the specified string.

# Special Characters

- \ (Backslash) escapes metacharacters or signals a particular sequence.
- ^ (Caret) is used to match the beginning of a string.
- . (Dot) matches all characters except a newline.
- [] (Square Brackets) matches one of the characters within brackets.
- \$ (Dollar Sign) matches the end of the string.

# Special Sequences

- `\s` to match a whitespace character.
- `\w` to match an alphanumeric character.
- `\W` to match a non-alphanumeric character.
- `\d` to match a digit.
- `\D` to match a non-digit character.
- `\S` to match a non-whitespace character.

# Quantifiers

- `?` to match 0 or 1 repetition of the preceding pattern.
- `{n,}` to match n or more repetitions of the preceding pattern.
- `{n}` to match n repetitions of the preceding pattern.
- `{n,m}` to match between n and m repetitions of the preceding pattern.
- `*` to match 0 or more occurrences of the preceding pattern.
- `+` to match 1 or more occurrences of the preceding pattern.

# Metacharacters in RegEx

## 1. \ – Backslash

- Backslash (\), known as the escape character, tells Python to treat a special character (like \$, ., or \*) as a normal character. It's used to escape metacharacters or define special sequences like \d or \s.

```
import re

text = "Price is $100"
match = re.search(r"\$", text) # Match the $ character using escape

if match:
    print("Matched:", match.group())
```

Output:

```
Matched: $
```

## 2. [] – Square Brackets

- Regular expressions in Python use square brackets [] to define a character set or range.
- Use - to specify a range of characters.
- Use ^ to invert (negate) the set.

Thus:

- [0,3] is the same as [0123]
- [b-d] is the same as [bcd]
- [^0-3] matches anything except 0, 1, 2, or 3
- [^b-d] matches anything except b, c, or d

Example:

```
import re

pattern = r'[a-zA-Z]' # Match any lowercase or uppercase English letter
string = "123 abc XYZ"

matches = re.findall(pattern, string) # Find all matching characters
print("Matches found:", matches)
```

Output:

```
Matches found: ['a', 'b', 'c', 'X', 'Y', 'Z']
```

### 3. ^ – Caret

- The caret symbol (^) matches the beginning of a string. This means we can check if a string begins with the mentioned character.
- ^h matches strings like "home", "house", "host"
- ^ho matches strings starting with "ho", like "home" or "host"

```
import re

pattern = r'^Hello' # Match if the string starts with 'Hello'
string = "Hello, world!"

match = re.match(pattern, string)

if match:
    print("Match found:", match.group())
else:
    print("No match found.")
```

Output:

```
Match found: Hello
```

#### 4. \$ – Dollar

- The dollar symbol (\$) is used in regular expressions to match the end of a string. It ensures that the string ends with the specified character or pattern.

```
import re

pattern = r'world!$' # Match 'world!' at the end of the string
string = "Hello, world!"

match = re.search(pattern, string)

if match:
    print("Match found:", match.group())
else:
    print("No match found.")
```

## 5. . – Dot

- The dot (.) matches any single character except a newline. It acts as a wildcard in patterns.
- c.d matches "cad", "cbd", "c9d", etc.
- .. checks if the string contains at least two characters

```
import re

pattern = r'a.c' # Match 'a', any character, then 'c'
string = "abc a1c a2c"

matches = re.findall(pattern, string)
print("Matches found:", matches)
```

Output:

```
Matches found: ['abc', 'a1c', 'a2c']
```

## 6. | – Or

- The pipe symbol (|) is used to match either the pattern before or after it. It helps us search for multiple alternatives within a string.
- c|d matches either 'c' or 'd'
- (cat|dog) matches either 'cat' or 'dog'

```
import re # Importing the regex module

pattern = r'cat|dog' # Matches either 'cat' or 'dog'
string = "I have a cat and a dog."

matches = re.findall(pattern, string)
print("Matches found:", matches)
```

Output:

```
Matches found: ['cat', 'dog']
```

## 7. ? – Question Mark

- The question mark (?) symbol is a quantifier in regular expressions in Python, showing that the preceding element must be matched zero or one time. We can specify the element as optional, which means it can occur once or not at all.
- `cd?a` matches "ca" and "cda" (the 'd' is optional)

```
import re
```

```
pattern = r'cd?a' # Matches 'c' followed by 0 or 1 'd', then 'a'  
string = "ca cda cdba"
```

```
matches = re.findall(pattern, string)  
print("Matches found:", matches)
```

Output:

```
Matches found: ['ca', 'cda']
```

## 8. \* – Star

- The asterisk (\*) is a quantifier that matches zero or more repetitions of the character or group before it.
- `cd*a` matches "ca", "cda", "cdda", "cddda", etc.

```
import re

pattern = r'cd*a' # 'c' followed by 0 or more 'd', then 'a'
string = "ca cda cddda cdba"

matches = re.findall(pattern, string)
print("Matches found:", matches)
```

Output:

```
Matches found: ['ca', 'cda', 'cddda']
```

## 9. + – Plus

- The plus (+) is a quantifier in regular expressions. It matches one or more repetitions of the character or group before it.
- `cd+a` matches "cda", "cdda", "cddda", etc.
- It does not match "ca" (because at least one 'd' is required)

```
import re

pattern = r'cd+a' # 'c' followed by 1 or more 'd', then 'a'
string = "ca cda cddda cdba"

matches = re.findall(pattern, string)
print("Matches found:", matches)
```

Output:

```
Matches found: ['cda', 'cddda']
```

## 10. {m, n} – Braces

- Braces {m,n} match at least m and at most n repetitions of the preceding character or group.
- For example, `c{1,3}a` matches patterns like "ca", "cca", or "ccca", but not "a" or "ccccca".

```
import re

pattern = r'c{1,3}a'          # Match 'c' repeated 1 to 3 times followed by 'a'
string = "a ca cca ccca cccca ab"

matches = re.findall(pattern, string) # Find all matches in the string
print("Matches found:", matches)
```

Run

Output:

```
Matches found: ['ca', 'cca', 'ccca']
```

## 11. (<regex>) – Group

- We use parentheses () in regular expressions to group sub patterns.

```
import re

pattern = r'(ab)+'          # Group 'ab' and match it one or more times
string = "abab abc ababab"

matches = re.findall(pattern, string) # Find all matching groups
print("Matches found:", matches)
```

Output:

```
Matches found: ['ab', 'ab', 'ab']
```

# Common RegEx functions

## 1. re.findall()

- The findall() function searches a string for all parts that match a pattern and returns them as a list.
- Strings are scanned left to right.

```
import re

# Pattern to find all words in the string
pattern = r'\w+'
text = "Python 3.12 is awesome!"

# Find all words or alphanumeric sequences
matches = re.findall(pattern, text)

# Display the matches
print("Matches found:", matches)
```

Output:

```
Matches found: ['Python', '3', '12', 'is', 'awesome']
```

## 2. re.search()

- The search() function looks through a string and finds the first match of the pattern.
- It stops at the first match and returns details about it. If nothing matches, it returns None.

```
import re

# Pattern to search for the word 'Python'
pattern = r'Python'
text = "I love Python and Python is powerful."

# Search for the first match
match = re.search(pattern, text)

# Check and print the result
if match:
    print("First match found:", match.group()) # Output: Python
else:
    print("No match found.")
```

Output:

```
First match found: Python
```

### 3. re.compile()

- The compile() function converts a Python regular expression pattern into a reusable object.
- This object can then be used multiple times to match or search strings, saving time and improving performance.

```
import re

# Compile the pattern once for reuse
pattern = re.compile(r'\d+') # Matches one or more digits

# Use the compiled pattern to search in strings
match1 = pattern.search("Order number: 12345")
match2 = pattern.search("Invoice: 9876")

# Print matches
print("First match:", match1.group())
print("Second match:", match2.group())
```

Output:

```
First match: 12345
Second match: 9876
```

## 4. re.escape()

- The `escape()` function adds a backslash (`\`) before every special character in a string.
- We use it when we want to treat all characters as plain text, even if they are RegEx symbols.

Syntax:

```
re.escape(string)
```

Example:

```
import re

text = "Price is $40.99 (tax included)."
```

# Escape all special characters in the string

```
escaped_text = re.escape(text)

print("Escaped string:", escaped_text)
```

Output:

```
Escaped string: Price\ is\ \$40\.99\ \(tax\ included\)\. 
```

## 5. re.split()

- The split function splits the string by the pattern occurrences.
- Once the pattern is found, it returns the remaining characters from the string as a list after splitting.

Syntax:

```
re.split(pattern, string, maxsplit=0, flags=0)
```

- **Pattern:** The regex pattern to split on.
- **string:** The string to be split.
- **maxsplit:** (Optional) Maximum number of splits. Defaults to 0 (no limit).
- **flags:** (Optional) Regex flags like re.IGNORECASE.

```
import re

text = "apple, banana; mango orange"
# Split on comma, semicolon, or space
result = re.split(r'[;, ]+', text)

print("Split result:", result)
```

Output:

```
Split result: ['apple', 'banana', 'mango', 'orange']
```

## 6. re.sub()

- The `re.sub()` function searches for a regex pattern in a string and replaces it with a specified substring. It's commonly used to clean or modify text.

Syntax:

```
re.sub(pattern, repl, string, count=0, flags=0)
```

- **pattern:** The regex pattern to search.
- **repl:** The replacement string.
- **string:** The input string.
- **count:** (Optional) How many matches to replace. The default is 0 (replace all).
- **flags:** (Optional) Regex flags like `re.IGNORECASE`.

```
import re

text = "blue sky and blue ocean"
# Replace "blue" with "green"
new_text = re.sub(r'blue', 'green', text)

print("Updated text:", new_text)
```

Output:

```
Updated text: green sky and green ocean
```

## 7. re.subn()

- The subn() function and sub() functions are the same. However, they differ in the way they provide output.
- Unlike sub(), which returns only a string, the subn() function returns a tuple with total replacement counts and a new string.

### Syntax:

```
re.subn(pattern, repl, string, count=0, flags=0)
```

- **pattern:** The regex pattern to match.
- **repl:** The string to replace the match with.
- **string:** The original string.
- **count:** (Optional) Limits the number of replacements.
- **flags:** (Optional) Regex flags like re.IGNORECASE.

```
import re

text = "apple banana apple orange apple"
# Replace "apple" with "fruit"
result = re.subn(r'apple', 'fruit', text)

print("Updated text:", result[0]) # New string
print("Replacements made:", result[1]) # Number of replacements
```

Output:

```
Updated text: fruit banana fruit orange fruit
Replacements made: 3
```

# Match method Vs Search method

- The `match()` and `search()` methods are both used to search for a pattern in a string using RegEx in Python, but they have different behaviors:
- `match()` method: This method searches for the pattern at the beginning of the string. If the pattern is found at the beginning of the string, it returns a match object. Otherwise, it returns `None`.
- `search()` method: This method searches for the pattern in the entire string. If the pattern is found anywhere in the string, it returns a match object. Otherwise, it returns `None`.

```
import re
```

```
string = "The quick brown fox jumps over the lazy dog"
```

```
pattern = r"fox"
```

```
# Using match() method
```

```
match_object = re.match(pattern, string)
```

```
print(match_object)
```

```
# Using search() method
```

```
match_object = re.search(pattern, string)
```

```
print(match_object)
```

- When we print the match object returned by each method, we can see that the `match()` method returns `None` because the pattern is not found at the beginning of the string, while the `search()` method returns a match object because the pattern is found later in the string.
- In summary, the `match()` method is useful when you want to search for a pattern at the beginning of a string, while the `search()` method is useful when you want to search for a pattern anywhere in a string.

# Finditer Method

- `re.finditer()` method in [Python](#) is to search for all matches of a pattern in a [string](#) and return them as an iterator.

```
import re

text = "GFG, O, B, GFG, O"
pattern = "GFG"

# Find all occurrences of "apple" in the string
matches = re.finditer(pattern, text)

for match in matches:
    print(match.start(), match.group())
```

## Syntax of re.finditer()

*re.finditer(pattern, string)*

### Output

0 GFG

11 GFG

```
import re

text = "gfg, gg, geeksforgeeks, gfg, gg"
pattern = "gfg|geeksforgeeks"

matches = re.finditer(pattern, text)

for match in matches:
    print(match.start(), match.group())
```

## Output

```
0 gfg
9 geeksforgeeks
24 gfg
```

```
import re
```

```
# Sample text containing dates in DD-MM-YYYY format
```

```
text = """
```

```
The meeting will be held on 12-05-2024.
```

```
Another important date is 23-06-2023.
```

```
Remember, the report is due on 07-09-2024.
```

```
Also, we have a holiday on 25-12-2024.
```

```
"""
```

```
# Define the regular expression to match dates in the DD-MM-YYYY format
```

```
dp = r'\b\d{2}-\d{2}-\d{4}\b'
```

```
# Use re.finditer() to find all matches
```

```
matches = re.finditer(dp, text)
```

```
# Loop through the matches and print their details
```

```
for match in matches:
```

```
    print(f"Found date: {match.group()} at position {match.start()} to {match.end()}")
```

## Output

Found date: 12-05-2024 at position 29 to 39

Found date: 23-06-2023 at position 67 to 77

Found date: 07-09-2024 at position 110 to 120

Found date: 25-12-2024 at position 149 to 159

# Text Processing

- Text processing is a fundamental step in Natural Language Processing (NLP). It involves converting raw text into structured formats for computational models.
- **Word** : A meaningful linguistic unit (e.g., 'apple').  
**Token** : The occurrence of a word in a text. For example, 'apple apple' has two tokens.  
**Vocabulary** : The set of unique tokens extracted from a text or corpus.  
**Corpus** : A large collection of texts used for NLP tasks.

Example:

Text: 'I love NLP. NLP is amazing.'

Tokens: ['I', 'love', 'NLP', 'NLP', 'is', 'amazing']

Vocabulary: {'I', 'love', 'NLP', 'is', 'amazing'}

# Sentiment Classification

- Sentiment classification involves predicting the sentiment (positive/negative) of reviews.

Steps:

1. Download Dataset : Yelp reviews dataset.
2. Data Preparation : Using NumPy, pandas, re, and Counter.
3. Build Vocabulary: From tokens, Data-Frames, or corpus.
4. Encoding: One-hot encoding or other methods.
5. Train-Test Split: Divide data for training and evaluation.
6. Feature Computation: Convert text into numerical vectors.
7. Confusion Matrix: Evaluate predictions.
8. Analysis: Identify accuracy, precision, recall, F1-score.

# Tokenization in spaCy

Tokenization is the process of splitting text into words, phrases, or symbols.  
spaCy provides efficient tokenization for multiple languages.

Example:

```
import spacy
nlp = spacy.load('en_core_web_sm')
doc = nlp('I love NLP!')
for token in doc:
    print(token.text)
```

Output: I, love, NLP, !

# Language Independent Tokenization

- Tokenization is the process of splitting text into smaller meaningful units called **tokens**. These tokens are later used by Natural Language Processing (NLP) systems for tasks like translation, summarization, classification, etc.
- A **language-independent tokenizer** aims to handle text across different languages without requiring handcrafted rules for each language. Instead, it applies statistical or data-driven methods to break down text into tokens.

# Types of Tokenization

- 1. Word Tokenization
- 2. Character Tokenization
- 3. Sub-word Tokenization

# 1. Word Tokenization

- **Definition:** Splits text into words using spaces or punctuation as delimiters.
- **Example:**  
Text: *"Tokenization is important."*  
Tokens: ["Tokenization", "is", "important"]

## Advantages

- Simple and intuitive.
- Works well for space-separated languages (English, French, etc.).

# Problems with Word Tokenization

- **Language dependency:** Does not work well for languages like Chinese, Japanese, Thai (no spaces between words).
- **Out-of-Vocabulary (OOV) problem:** Unseen or rare words are not handled properly. For example, “tokenizers” may not be in the vocabulary.
- **Morphologically rich languages** (e.g., Turkish, Finnish) create many word forms from a single root, leading to large vocabularies.

## 2. Character Tokenization

- **Definition:** Splits text into individual characters.
- **Example:**  
Text: *"Cat"*  
Tokens: ["C", "a", "t"]

### Advantages

- Completely language-independent.
- No OOV problem — any word can be represented as a sequence of characters.
- Useful for handling typos or unknown words.

# Drawbacks of Character-Based Tokenizer

- Sequences become very long, increasing computation.
- Loses semantic meaning — characters alone may not carry useful information.
- Models need to learn long-range dependencies to understand meaning.

# 3. Sub-word Tokenization

- Sub-word Tokenization
- Definition: Splits words into smaller units like prefixes, suffixes, or meaningful chunks.
- Example:  
Word: *"tokenization"*  
Tokens: ["token", "ization"]

## Advantages

- Reduces OOV problem: Unseen words can be broken into sub-words.
- Vocabulary size is smaller compared to word tokenization.
- Balances between **word** and **character** tokenization.

# Problems with Sub-word Tokenization

- Still language-dependent if not carefully designed.
- May produce unnatural splits (e.g., “run-ning” instead of “running”).
- Requires a preprocessing step (training a tokenizer).

# Byte Pair Encoding (BPE)

- **Definition:** A data compression technique adapted for sub-word tokenization. It iteratively merges the most frequent pairs of bytes/characters into a new symbol.
- **Steps:**
  - Start with a vocabulary of individual characters.
  - Count frequent adjacent character pairs.
  - Merge the most frequent pair into a new token.
  - Repeat until the desired vocabulary size is reached.

## Example:

Training data: ["low", "lower", "newest", "widest"]

- Start with characters: l, o, w, e, r, n, s, t, d, ...
- Merge most frequent pair "lo" → new token lo
- Merge "low" → new token low
- Continue until sub-words like ["low", "er", "new", "est", "wid", "est"] emerge.

## Advantages of BPE

- Handles rare words by breaking them into sub-words.
- Reduces vocabulary size while maintaining coverage.
- Widely used in models like GPT, BERT, RoBERTa.

## Drawbacks of BPE

- May split words in unnatural ways (hurting interpretability).
- Struggles with some languages where morphemes are complex (e.g., agglutinative languages).

# String Matching and Spelling Correction

## String Matching and Spelling Correction

- In NLP and text processing, one common task is to **match two strings** (e.g., query vs dictionary word) and find how similar they are. For spelling correction, we need to measure **how far a misspelled word is from a correct word**.
- The **Minimum Edit Distance** algorithm is the foundation for this.

# Minimum Edit Distance

- **Definition:** The minimum number of edit operations required to transform one string into another.
- **Edit operations:**
  - **Insertion** → add a character.
  - **Deletion** → remove a character.
  - **Substitution** → replace one character with another.

- **Example:**

Transform "kitten" → "sitting"

- kitten → sitten (substitute 'k' → 's')
- sitten → sittin (substitute 'e' → 'i')
- sittin → sitting (insert 'g')
- 

**Minimum Edit Distance = 3**

# Dynamic Programming Approach

The edit distance is computed using a **DP table** that stores intermediate results.

- **Steps:**

1. Create a table  $D$  of size  $(m+1) \times (n+1)$  where  $m$  = length of string1,  $n$  = length of string2.

2. Initialize base cases:

- $D[i][0] = i$  ( $i$  deletions needed to convert string of length  $i$  to empty string)
- $D[0][j] = j$  ( $j$  insertions needed to convert empty string to length  $j$ )

# Steps

3. Fill the table using recurrence relation:

Where:

- $D[i-1][j] + 1 \rightarrow$  deletion
- $D[i][j-1] + 1 \rightarrow$  insertion
- $D[i-1][j-1] + 1 \rightarrow$  substitution

$\cdot$  A B C A B  
 E A C B

$\rightarrow$  E A ~~B~~ C ~~A~~ <sub>3</sub> <sub>2</sub> 1

E A C

$\rightarrow$  E A C B

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
|   |   | A | B | C | A | B |
|   | 0 | 1 | 2 | 3 | 4 | 5 |
| E | 1 | 1 | 2 | 3 | 4 | 5 |
| A | 2 | 1 | 2 | 3 | 3 | 4 |
| C | 3 | 2 | 2 | 2 | 3 | 4 |
| B | 4 | 3 | 2 | 3 | 3 | 3 |

↑

Minimum Edit Distance = 5

|     |  |  | ''' |  | e |  | x |  | e |  | c |  | u |  | t |  | i |  | o |  | n |  |
|-----|--|--|-----|--|---|--|---|--|---|--|---|--|---|--|---|--|---|--|---|--|---|--|
| ''' |  |  | 0   |  | 1 |  | 2 |  | 3 |  | 4 |  | 5 |  | 6 |  | 7 |  | 8 |  | 9 |  |
| i   |  |  | 1   |  | 1 |  | 2 |  | 3 |  | 4 |  | 5 |  | 6 |  | 6 |  | 7 |  | 8 |  |
| n   |  |  | 2   |  | 2 |  | 2 |  | 3 |  | 4 |  | 5 |  | 6 |  | 7 |  | 7 |  | 7 |  |
| t   |  |  | 3   |  | 3 |  | 3 |  | 3 |  | 4 |  | 5 |  | 5 |  | 6 |  | 7 |  | 8 |  |
| e   |  |  | 4   |  | 3 |  | 4 |  | 3 |  | 4 |  | 5 |  | 6 |  | 6 |  | 7 |  | 8 |  |
| n   |  |  | 5   |  | 4 |  | 4 |  | 4 |  | 4 |  | 5 |  | 6 |  | 7 |  | 7 |  | 7 |  |
| t   |  |  | 6   |  | 5 |  | 5 |  | 5 |  | 5 |  | 5 |  | 5 |  | 6 |  | 7 |  | 8 |  |
| i   |  |  | 7   |  | 6 |  | 6 |  | 6 |  | 6 |  | 6 |  | 6 |  | 5 |  | 6 |  | 7 |  |
| o   |  |  | 8   |  | 7 |  | 7 |  | 7 |  | 7 |  | 7 |  | 7 |  | 6 |  | 5 |  | 6 |  |
| n   |  |  | 9   |  | 8 |  | 8 |  | 8 |  | 8 |  | 8 |  | 8 |  | 7 |  | 6 |  | 5 |  |

# Applications in Spelling Correction

- When a user types "recieve", compare it with dictionary words.
- Calculate **edit distance** with candidates:
  - receive → distance = 1
  - relieve → distance = 2
- The word with **minimum edit distance** is suggested as the correction.

**THANKYOU**